

DTSG: A Dynamic Temporal State Graph for Agent Memory

ACM Research Summer Internship 2026

Submitted By

Sneha (*Complete Enrollment Number*)
(*official email*)

under the supervision of

Name of Faculty
Designation

Department of Information Technology
Indira Gandhi Delhi Technical University for Women

This report is dedicated to ...

STUDENT UNDERTAKING

Dated:

We hereby undertake that the work titled “*DTSG: A Dynamic Temporal State Graph for Agent Memory*”, submitted as part of this Summer Internship Report during June–July 2026, under the guidance of (*name of faculty*), is our original work.

The concepts, analysis, observations, implementation, results, and project narration presented in this report are our own. This report has been written in our own words and has not been copied, wholly or in part, from any other source. Wherever the ideas, data, figures, text, or work of others have been used, they have been appropriately acknowledged and cited. While we acknowledge that we have used AI-assisted tools (*mention names*) for improving language, grammar, spelling, formatting, clarity, or readability.

We understand that any form of plagiarism, falsification, fabrication, unauthorized AI-generated academic content, or any other act of academic misconduct, whether discovered now or in the future, shall be solely our responsibility. In such an event, we acknowledge that the University may initiate appropriate disciplinary action in accordance with its rules and regulations, and we agree to abide by the decisions of the competent authorities.

Student Signatures

Student Names

Date-of-Submission, New Delhi

**DEPARTMENT OF INFORMATION TECHNOLOGY
INDIRA GANDHI
DELHI TECHNICAL UNIVERSITY FOR WOMEN
KASHMERE GATE, DELHI – 110006**

Dated:

CERTIFICATE

This is to certify that the work titled “*DTSG: A Dynamic Temporal State Graph for Agent Memory*” submitted by (*student names*) in this Summer Internship Report during June–July 2026 was done under my guidance and supervision.

This work is their original work and has not been submitted anywhere else for the award of any credits to the best of my knowledge. The work is satisfactory for the award of Summer Internship credits for the said period.

Name and Signature of Faculty Advisor(s)

Designation

Department of Information Technology

Indira Gandhi Delhi Technical University for Women

ACKNOWLEDGEMENT

To be written by student . . .

Student Name

Contents

1	Introduction	2
2	Related Work	3
3	Dataset Description	7
4	Proposed Methodology	9
4.1	Data Pre-processing	9
4.2	Proposed Workflow / Architecture	10
4.2.1	The write pipeline	10
4.2.2	Temporal retrieval	12
4.2.3	The timeline view	13
5	Experiment Design	13
6	Results & Observations	15
7	Future Work and Conclusion	17

DTSG: A Dynamic Temporal State Graph for Agent Memory

Student Name (enrolment number, official email), ...

Summer Internship 2026

Table 1: Contributions of team members.

Student Name	Enroll Number	Contributions
XYZ	0xx010120xx	(1) Designed and applied the Supabase schema (migrations 0001–0004), including the append-only triggers on events ; (2) implemented the extraction and normalization stage (Section 4.1); (3) wrote Sections 1 and 3.
ABC	0xx010120xx	(1) Implemented the conflict classifier and the write pipeline (Section 4.2); (2) built the temporal re-ranker and the benchmark harness; (3) wrote Sections 5 and 6.

Note: the entire base paper has to be read by all team members.

Abstract

Conversational AI agents forget. Each session begins from an empty state, and the standard remedy — retrieval-augmented generation over a vector store of past messages — treats memory as a flat, timeless pile of text. This is a poor fit for what people actually tell an agent, because those statements *change*: a user who once lived in Delhi later lives in Berlin, and a store that has both sentences and no notion of which one is still true will answer “where do I live?” with whichever sentence embeds closer to the question. Prior systems address this partially. Naive RAG pipelines keep every utterance and rank by similarity alone; agent-memory systems such as MemGPT manage context through paging but do not model fact validity; more recent graph-based memories such as Zep and Mem0 introduce structure and invalidation, but their conflict handling is largely internal and their retrieval policies are hard to ablate independently of the underlying language model. We propose DTSG, a Dynamic Temporal State Graph that splits memory into an *immutable* event log and a *mutable* graph of subject–predicate–object facts, each carrying a status, a validity interval and explicit supersede links. Facts are never deleted; a contradicted fact is marked **EXPIRED**, stamped with **valid_until** and linked to its replacement, so history remains queryable. Retrieval multiplies three terms — cosine similarity, a status weight evaluated at query time, and an exponential recency decay — which lets the same store answer both “what is true now” and “what was true in March”. On a benchmark of retrieval cases where the stale fact is the equal-or-better textual match, DTSG selects the correct current fact in 3/3 scored cases while a cosine-only baseline selects it in 0/3, and DTSG never returns a stale fact as its top hit (0/4 vs. 3/4 for the baseline). The system is implemented end to end (Postgres/pgvector, FastAPI, React) with 103 passing tests.

1 Introduction

Large language models are stateless between calls. Everything an agent “knows” about a user within a session sits in its context window, and when that window ends the knowledge ends with it. The dominant workaround is retrieval-augmented generation (RAG) [10]: past messages are embedded into a vector store and, at question time, the nearest neighbours of the query are pasted back into the prompt. This works well for what it was designed for — looking things up in a corpus of documents that do not contradict one another.

Personal memory is not that corpus. What a user says about themselves is a sequence of assertions about a world that keeps moving. “I live in Delhi” said in July 2025 and “I just moved to Berlin” said in May 2026 are not two facts; they are one fact with two values, and only the second is currently true. A vector store holds both sentences with no relation between them. Worse, the two sentences are near-identical in embedding space — both are about the speaker’s place of residence — so a query like “where do I live?” retrieves them with nearly the same score, and which one lands on top is decided by noise rather than by truth. The failure is quiet: the agent does not say “I am unsure”; it confidently states the stale answer. Benchmarks of long-horizon conversational memory report exactly this pattern, with accuracy collapsing on questions whose answer was updated during the conversation [8, 7]. The same brittleness under temporal drift has been documented for question-answering over changing facts more broadly [12, 13, 14].

Others have attacked this from several directions, and each leaves something open. MemGPT [3] treats the context window like virtual memory and pages information in and out, which solves *capacity* but not *validity*: a paged-in fact is no more or less true than any other. Generative Agents [4] scores memories by an additive combination of recency, importance and relevance, which acknowledges time but has no representation of contradiction, so a superseded memory merely decays rather than being marked wrong. MemoryBank [5] applies an Ebbinghaus-style forgetting curve, which again is decay without invalidation. Closest to our work, Zep [1] maintains a temporally-aware knowledge graph with bi-temporal edge validity and invalidation, and Mem0 [2] has an LLM decide among ADD/UPDATE/DELETE operations on extracted memories. Both report strong results, but both couple the memory policy to a full agent stack, which makes it difficult to see how much of the gain comes from the temporal ranking itself; Mem0’s DELETE operation additionally removes information, so questions about what a user *used* to believe become unanswerable.

We propose DTSG (Dynamic Temporal State Graph), a memory engine built around a single commitment: **nothing is ever deleted, and everything is timestamped**. Memory is split into two stores with different mutability rules. The **events** table is an append-only log of raw user messages, with immutability enforced by database triggers rather than by convention. The **memories** table is a mutable graph of subject–predicate–object facts, each carrying a **status** (ACTIVE/EXPIRED), a confidence, a validity interval and **supersedes/superseded_by** links. On each message, facts are extracted and normalized, candidate conflicts are found by a two-armed search (exact subject–predicate plus vector similarity), and a language model classifies the new fact as REINFORCE, ADDITIVE or SUPERSEDE. Retrieval then scores each candidate as the product of cosine similarity, a status weight *evaluated at the query’s reference time*, and an exponential recency decay — a formulation that makes “what did I say before X” a ranking question over one store rather than a second store.

The design pays off exactly where plain similarity fails. On a purpose-built benchmark of cases in which the outdated fact is the equal-or-better textual match, the cosine-only baseline puts a stale fact on top in 3 of 4 cases and gets 0 of 3 scored cases right; DTSG gets 3 of 3 right and never tops a stale fact. In the controlled “Delhi → Berlin” scenario where both facts have *identical* cosine similarity (1.000), so a vector store cannot possibly distinguish them, DTSG scores Berlin at 0.9622 against Delhi at 0.1443 — and when the same query is asked **as_of** a date before the move, the order inverts, with Delhi at 1.0000 and Berlin at 0.1500.

The main contributions of this work are:

- We design and implement a memory engine that separates an **immutable event log** from a **mutable temporal state graph**, with append-only semantics enforced by database triggers and a scoped, auditable escape hatch for lawful erasure — to the best of our knowledge, prior agent-memory systems enforce this by convention only.
- We formulate retrieval as a **decomposable product** of similarity, status weight and recency decay in which the status term is evaluated at a caller-supplied reference time. This makes present-tense and point-in-time questions the same query with a different parameter, and every response returns the three terms separately so a ranking can be debugged rather than merely observed.
- We isolate the contribution of the ranking policy with a **model-free benchmark**: cases carry their own similarity scores, so the comparison between DTSG and naive RAG does not depend on the quality of any embedding model. DTSG selects the correct current fact in 3/3 scored cases versus 0/3 for the baseline, and returns a stale top hit in 0/4 cases versus 3/4.
- We identify the ADDITIVE-vs-SUPERSEDE decision (whether a predicate is single- or multi-valued) as the judgement that governs memory correctness, argue that it is not derivable from the schema, and give an asymmetric error policy that biases toward the recoverable failure.

The rest of this report is organized as follows. Section 2 surveys related work on agent memory, temporal knowledge graphs and temporal question answering. Section 3 describes the data the system operates on and the evaluation cases we constructed. Section 4 presents the proposed methodology — extraction and normalization in Section 4.1 and the full architecture in Section 4.2. Section 5 describes the experiment design, Section 6 reports and discusses results, and Section 7 covers limitations, future work and conclusions.

2 Related Work

Base paper. We selected **Zep: A Temporal Knowledge Graph Architecture for Agent Memory**, by Rasmussen et al. [1], as our base paper. Zep built a memory layer around Graphiti, a temporally-aware knowledge graph engine that ingested conversational messages and structured data into a graph of episodes, semantic entities and communities. Its central idea, and the one this project adopted, was *bi-temporal* edge validity: each relationship recorded both when it was ingested and the interval over which it held in the world, so a contradicted edge was invalidated by setting its expiry rather than by deleting it. The authors evaluated on the Deep Memory Retrieval task and on LongMemEval, reporting accuracy improvements over a full-context baseline together with large reductions in response latency, because the agent read a compact retrieved subgraph rather than an entire conversation history. We follow Zep’s commitment to invalidation-without-deletion, and depart from it in two ways: our state graph is a flat subject–predicate–object store rather than a hierarchical entity/community graph, and our retrieval score is an explicit closed-form product whose terms can be swept per request, which is what makes the ablation in Section 6 possible.

Important papers. Table 2 compares the works that address the same problem setting — persistent, updatable memory for conversational agents. Mem0 [2] was the closest competitor in spirit: it extracted salient memories from a dialogue and used an LLM to choose an ADD, UPDATE, DELETE or NOOP operation against the existing store, with a graph variant

(Mem0^g) adding entity relations. The authors reported substantial accuracy gains over full-context and RAG baselines on LoCoMo along with order-of-magnitude reductions in token cost. Our classifier is structurally similar, but our resolution set is REINFORCE/ADDITIVE/SUPERSEDE with no delete, because deletion is precisely what makes “what did I say before” unanswerable. MemGPT [3] introduced an operating-system analogy for context management, with the model issuing function calls to page data between a small “main context” and external storage; it solved context capacity but left fact validity untouched. Generative Agents [4] introduced a retrieval score combining recency (exponential decay), importance and relevance — an additive formulation — and a reflection mechanism that synthesised higher-level memories. LongMemEval [7] and LoCoMo [8] are the benchmarks these systems are measured on; both explicitly include a “knowledge update” question category, and both report that it is among the hardest, which is direct evidence for the problem this project targets.

Other papers. Table 3 lists work on the same underlying problem but over different data or with different machinery. HippoRAG [9] drew on the hippocampal indexing theory of human memory, building a knowledge graph over a corpus and running personalized PageRank to perform multi-hop retrieval in a single step; A-MEM [6] proposed agentic memory notes that link and evolve themselves in a Zettelkasten-like structure. MemoryBank [5] applied an Ebbinghaus forgetting curve so that unreinforced memories faded and restated ones strengthened — our `last_reinforced_at` column serves the same purpose in the recency term. On the temporal knowledge graph side, Know-Evolve [16] modelled facts as a temporal point process, TA-DistMult [17] encoded timestamps into the relation embedding, RE-NET [18] modelled event sequences autoregressively for future-time prediction, and the surveys of Cai et al. [19] organise the field into interpolation (completing the past) and extrapolation (predicting the future) settings; DTSG sits in the interpolation setting but is written rather than learned. For temporal question answering, TempLAMA [13] showed that a model’s factual recall degrades as facts change over time and proposed temporally-scoped training; TimeQA [14] and StreamingQA [15] provide datasets where the correct answer depends on the timestamp of the question, which is the read-side analogue of our `as_of` mode. FreshLLMs [12] quantified how quickly parametric knowledge goes stale and argued for retrieval over fast-changing facts. A parallel line edits the model rather than an external store: ROME [20] located and edited individual factual associations in the weights, and MEMIT [21] scaled this to thousands of edits; both change what the model believes but leave no queryable record of what it believed before. Self-RAG [11] taught a model to decide when to retrieve and to critique what it retrieved, which is complementary — it improves *whether* to look something up, not *which* version is current. Finally, work on immutable and bi-temporal data management in databases [22] predates all of this and supplies the vocabulary (valid time vs. transaction time) that our schema borrows.

Table 2: Comparative analysis between prior research works addressing the same problem setting as the base paper (persistent memory for conversational agents). Reported results are as claimed by the respective authors.

Author(s)	Brief description	Results
Rasmussen et al. [1] ¹ (base paper)	Zep: an agent memory service over Graphiti, a temporally-aware knowledge graph. Ingests messages and structured data as episodes, extracts entities and relations, and gives every edge a bi-temporal validity interval. Contradicted edges are invalidated by setting an expiry, never deleted, and retrieval combines semantic, keyword and graph traversal over the resulting subgraph.	Reports accuracy gains over a full-context baseline on Deep Memory Retrieval and LongMemEval, with latency reduced by roughly an order of magnitude, since the agent reads a small retrieved subgraph instead of a whole history.
Chhikara et al. [2] ²	Mem0: a two-phase memory pipeline. Extraction pulls salient memories from a conversation turn; an update phase asks an LLM to choose ADD, UPDATE, DELETE or NOOP against semantically similar stored memories. Mem0 ^g adds a graph representation of entities and their relations.	Reports large relative accuracy improvements over RAG and full-context baselines on LoCoMo, with roughly 90% lower token cost and substantially lower p95 latency than reading the full conversation.
Wu et al. [7] ³	LongMemEval: a benchmark isolating five long-term memory abilities in chat assistants, including <i>knowledge updates</i> , where a fact stated early is revised later and the system must answer with the revision. Also proposes a memory design space (indexing granularity, session decomposition).	Reports that commercial assistants and long-context LLMs lose a large fraction of their accuracy over sustained interactions, with knowledge-update questions among the weakest categories.
Maharana et al. [8] ⁴	LoCoMo: very long-term multi-session dialogues (hundreds of turns) annotated with single-hop, multi-hop, temporal and open-domain questions, used to evaluate memory in conversational agents.	Reports a wide gap between LLM performance and human performance, with temporal reasoning and knowledge-update questions the most difficult categories.
Packer et al. [3] ⁵	MemGPT: applies virtual-memory paging to the context window. The model itself issues function calls to move information between a bounded main context and external storage, producing the illusion of unbounded context.	Reports improved performance on multi-session chat and document QA relative to fixed-context baselines by retaining and recalling information across sessions.

Table 3: Prior research works related to our problem but using different datasets or operating on a different substrate.

Author(s)	Dataset	Brief description
Park et al. [4]	Simulated agent society (Smallville), authors' own	Generative agents keep a memory stream of observations and retrieve by an additive score of recency (exponential decay), importance and relevance, plus reflection into higher-level memories. Time is modelled as decay only; there is no representation of one memory contradicting another.
Zhong et al. [5]	Authors' own long-term dialogue set	MemoryBank stores dialogue summaries and updates memory strength using an Ebbinghaus-style forgetting curve, so restated memories persist and unused ones fade. Reinforcement-as-freshness is the mechanism our <code>last_reinforced_at</code> column reproduces.
Gutiérrez et al. [9]	MuSiQue, 2Wiki-MultiHopQA, HotpotQA	HippoRAG builds an open knowledge graph over a corpus and runs personalized PageRank from query-linked nodes, retrieving multi-hop evidence in one step instead of iterating.
Xu et al. [6]	LoCoMo	A-MEM builds Zettelkasten-style memory notes that generate their own attributes, link to related notes and update those neighbours when new memories arrive.
Dhingra et al. [13]	TempLAMA (Wikidata-derived)	Shows that a language model's factual recall degrades as the underlying facts change, and proposes temporally-scoped pretraining so predictions can be conditioned on a date.
Chen et al. [14]	TimeQA (Wikipedia + Wikidata)	Question answering where the answer depends on the time the question is about; constructed from time-evolving facts. This is the read-side analogue of our <code>as_of</code> retrieval mode.
Liška et al. [15]	StreamingQA (news)	A benchmark for adapting to a stream of new documents over time, measuring whether a model can answer questions about newly arrived facts without retraining.
Vu et al. [12]	FreshQA	Quantifies how quickly parametric knowledge becomes stale and shows that search-grounded prompting substantially improves accuracy on fast-changing questions.
Trivedi et al. [16]	GDELT, ICEWS	Know-Evolve models a temporal knowledge graph as a temporal point process, with entity embeddings that evolve as events occur.
García-Durán et al. [17]	ICEWS, YAGO15k	Encodes timestamps as token sequences appended to the relation, letting standard KG embedding models score time-annotated triples.
Jin et al. [18]	ICEWS, GDELT	RE-NET models the sequence of past graph snapshots autoregressively to predict facts at future timestamps (the extrapolation setting).
Cai et al. [19]	Survey	Organises temporal knowledge graph completion into interpolation and extrapolation settings and catalogues the standard benchmarks; useful for placing DTSG (written, interpolation) against learned approaches.
Meng et al. [20, 21]	CounterFact, zsRE	ROME locates and rewrites individual factual associations in model weights; MEMIT scales this to thousands of simultaneous edits. Neither leaves a queryable record of the superseded belief.
Lewis et al. [10]	Natural Questions, TriviaQA	The original retrieval-augmented generation formulation: a dense retriever feeds passages to a generator. It is the baseline whose timelessness this project is arguing against.
Asai et al. [11]	PopQA, PubHealth,	Self-RAG trains a model to decide when to retrieve and

3 Dataset Description

DTSG is a systems project rather than a model-training project, so “dataset” means two different things here and both are described.

(a) The operational corpus. At run time the system’s data is the user’s own conversation. Every message posted to `POST /api/chat` is appended verbatim to the `events` table, and the facts derived from it are written to `memories`. Nothing is collected from an external source and nothing is shared between users: every row in both tables is scoped by `user_id`. The two tables are described in Table 4 and their attributes in Table 5. Because the corpus is generated by use, its size at any moment is a property of the deployment rather than of the project; the counts in Table 4 are from the development database after manual exercise of the ingest path and are reported to characterise shape, not scale.

(b) The evaluation cases. To measure the retrieval policy we constructed a small, deliberately adversarial case set, held in `backend/scripts/run_benchmark.py` and extensible from a JSON file. Each case is a query plus a set of facts, where each fact carries its object, its status, how many days ago it became valid, how many days ago it expired (if it did), *and the cosine similarity it would receive against the query*. Supplying the similarity is the point of the design: it removes the embedding model from the experiment, so the only variable is the ranking policy. The selection rule for cases was explicit — every case is one where the stale fact is the equal-or-better textual match, i.e. exactly the situation in which cosine-only retrieval cannot be right by accident. Four cases were curated by hand (Table 6): three single-valued predicates where a replacement exists, and one multi-valued predicate included as a control against the opposite failure mode.

Table 4: Dataset description: the two stores and the evaluation case set.

Details	Count
Tables in the schema (<code>events</code> , <code>memories</code>)	2
Migrations applied	4
Columns in <code>events</code>	4
Columns in <code>memories</code>	13
Embedding dimensionality (<code>vector(1536)</code> , HNSW cosine)	1,536
Canonical predicates in the extraction vocabulary	15
Predicate synonyms mapped to canonical form	22
Benchmark cases (curated)	4
of which scored (single-valued, one correct answer)	3
of which control (multi-valued, either answer correct)	1
Facts per benchmark case	2
Automated tests over the pipeline	103

Important notes on subsetting and pre-processing. No subset of an external dataset is used, so no sampling procedure applies. The three scored cases were not sampled from a larger pool either; they were written to cover the three single-valued predicate families that dominate personal memory (residence, employer, role), with the similarity values chosen so that each case is adversarial in a different way — one where the stale fact is *more* similar (0.95 vs. 0.90), one where the two are *exactly tied* (0.93 vs. 0.93), and one where the gap is largest (0.97 vs. 0.88). This is a deliberate stress test and not a random sample of user queries, which is the principal

Table 5: Details of data attributes, with an example instance taken from the `memories` table. The `events` attributes are listed first.

Data attribute	Brief explanation	Example
<i>Table events — immutable log</i>		
<code>id</code>	Primary key of the message, uuid.	<code>a3f1...9c</code>
<code>user_id</code>	Owner of the message; every query is scoped by it.	<code>7b2e...41</code>
<code>raw_text</code>	The message exactly as sent, never rewritten.	"I just moved to Berlin"
<code>timestamp</code>	When the message was received.	2026-05-04 09:12:33+00
<i>Table memories — mutable state graph</i>		
<code>id</code>	Primary key of the fact, uuid.	<code>c81d...0a</code>
<code>subject</code>	Normalized subject; first person collapses to <code>user</code> .	<code>user</code>
<code>predicate</code>	Canonical snake_case relation.	<code>lives_in</code>
<code>object</code>	The value of the relation.	<code>Berlin</code>
<code>status</code>	ACTIVE or EXPIRED; never deleted.	ACTIVE
<code>confidence</code>	Extraction confidence, raised on reinforcement.	0.95
<code>embedding</code>	1536-d vector of "subject predicate object".	[0.013, -0.221, ...]
<code>source_event</code>	The event this fact was derived from (provenance).	<code>a3f1...9c</code>
<code>supersedes</code>	The closest fact this one replaced.	<code>9e40...b7</code>
<code>superseded_by</code>	The fact that replaced this one; the authoritative link.	<code>null</code> (still current)
<code>valid_from</code>	When the fact started being true.	2026-05-04 09:12:33+00
<code>valid_until</code>	When it stopped; <code>null</code> while current.	<code>null</code>
<code>last_reinforced_at</code>	When it was last restated (migration 0004).	2026-07-19 18:02:10+00

Table 6: The curated benchmark cases. In every case the stale fact is the equal-or-better textual match, so similarity alone cannot resolve it. The last case is a control: the predicate is multi-valued, so both facts are correct and either top hit is acceptable.

Query	Facts (object, status)	Sim.	Age (d)	Expected
where do I live?	Delhi (EXPIRED) / Berlin (ACTIVE)	0.95 / 0.90	400 / 100	Berlin
where do I work?	Acme (EXPIRED) / Globex (ACTIVE)	0.93 / 0.93	600 / 30	Globex
what is my job title?	Engineer (EXPIRED) / Staff Engineer (ACTIVE)	0.97 / 0.88	900 / 200	Staff Engineer
which languages do I speak?	English (ACTIVE) / Hindi (ACTIVE)	0.91 / 0.91	500 / 20	either (control)

threat to the external validity of the numbers in Section 6 and is stated as such there. Pre-processing applied to the operational corpus — extraction and normalization — is described in Section 4.1.

4 Proposed Methodology

4.1 Data Pre-processing

Raw user messages are unstructured, redundant and inconsistent in wording, and the state graph cannot be built from them directly. Pre-processing runs in two stages, and the second is load-bearing rather than cosmetic.

Stage 1 — Extraction. The message is sent to a language model (the `FAST` tier, `gemini-2.5-flash` by default) under a system prompt that asks for subject–predicate–object triples, using structured output so the response is schema-validated on arrival and cannot come back malformed. The prompt admits a triple only if it “would still be worth knowing in a week”, which filters out transient chatter. An empty fact list is a correct outcome, not an error: questions and greetings contain no durable facts.

Stage 2 — Normalization. The raw triples are folded into canonical form. This exists because conflict detection in Section 4.2 finds existing memories by matching (`subject`, `predicate`). If the model writes `lives_in` on Monday and `resides_in` on Tuesday, nothing matches, nothing supersedes, and the graph silently accumulates contradictory `ACTIVE` facts — a failure that is invisible until a query returns the wrong answer. Three operations are applied:

1. **Subject folding.** Every first-person term (`I`, `me`, `my`, `myself`, `mine`, `the user`, `speaker`) collapses to the single subject `user`. Without this, statements about the speaker become distinct graph nodes and never line up.
2. **Predicate casing.** Any predicate the model coins is lowercased and snake-cased, so a vocabulary that grows still grows in one shape.
3. **Synonym mapping.** 22 observed surface variants are mapped onto 15 canonical predicates through a lookup table — for example `resides_in`, `located_in`, `lives_at`, `based_in` and `moved_to` all become `lives_in`; `employed_at`, `works_for` and `employed_by` become `works_at`. This table is the project’s mechanism for absorbing model drift and is extended as new drift is observed.

Example. The message *“I’ve just moved to Berlin and I’m learning German”* produces, after Stage 1: (`I`, `moved_to`, `Berlin`, 0.95) and (`I`, `learning`, `German`, 0.9); and after Stage 2: (`user`, `lives_in`, `Berlin`, 0.95) and (`user`, `is_learning`, `German`, 0.9). The first now matches an existing (`user`, `lives_in`, `Delhi`) row on subject and predicate, which is what allows it to supersede that row.

Embedding. Each normalized fact is embedded for candidate search using `gemini-embedding-001` at 1536 dimensions, matching the `vector(1536)` column (the model supports this natively by Matryoshka truncation, so no migration is needed). The string embedded is “`subject predicate object`” with underscores replaced by spaces, since `lives_in` embeds better as *lives in*. Embeddings are *asymmetric*: stored facts use the `RETRIEVAL_DOCUMENT` role and search queries use `RETRIEVAL_QUERY`, which places them in the same space with better ranking; conflict detection compares a new fact against stored facts, so it uses the document role on both sides. If the embedding service fails, the fact is written with a null embedding rather than dropped —

candidate search degrades to exact matching, which loses recall, but losing the fact would be worse and the row can be backfilled.

4.2 Proposed Workflow / Architecture

Figure 1 shows the architecture. The system is a three-tier deployment — a React/TypeScript client on Vercel, a FastAPI service on Render, and Supabase Postgres with the `pgvector` extension — but the part that matters is the write pipeline and the read policy inside the service.

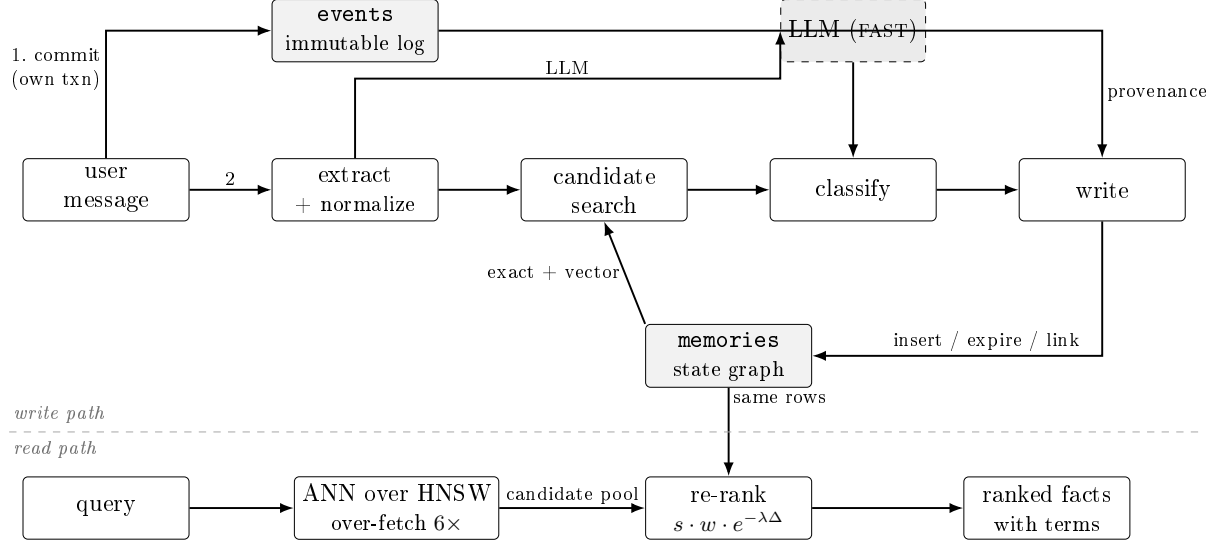


Figure 1: The DTSG architecture. The upper path is the write pipeline: the raw message is committed to the immutable event log *first, in its own transaction*, and only then is derived state computed — extraction, two-armed candidate search, LLM classification, and an atomic insert-and-expire. The lower path is retrieval: Postgres performs approximate nearest-neighbour search over the HNSW index and deliberately over-fetches, because top- K by cosine is not top- K by the full score; Python then applies the temporal re-ranking formula of Equation (1). Dashed nodes are language-model calls.

4.2.1 The write pipeline

The ingest path is

message $\rightarrow$ event (committed) $\rightarrow$ facts $\rightarrow$ candidates $\rightarrow$ classification $\rightarrow$ writes.

The event commits first, alone. Extraction and classification both call a language model, so both can fail on a rate limit or a timeout. If the whole flow shared one transaction, such a failure would roll back the raw message too — and the raw message is the one thing that cannot be reconstructed. Memories can always be rebuilt by replaying events; events cannot be rebuilt from anything. So the log commits in its own transaction and a downstream failure costs derived state, never history. Each fact is then applied in its own transaction, so one fact’s failure does not undo its predecessors’ writes.

Immutability is enforced, not intended. Migration 0002 places `BEFORE UPDATE` and `BEFORE DELETE` triggers on `events` that reject the operation for every role, service role included. Before this, “append-only” was a convention one careless query away from being false. Genuine deletions do eventually become necessary (an erasure request, a test teardown), so there is a deliberate, transaction-scoped escape hatch:

```

begin;
set local dtsg.allow_event_mutation = 'on';
delete from events where user_id = '...';
commit;

```

`set local` scopes the exemption to the transaction, so it cannot leak into the pooled connection’s next user, and application code never sets it. Erasing history stays possible, but never accidental.

Candidate search has two arms , because they fail in opposite directions. The **exact** arm matches on (`user_id`, `subject`, `predicate`) and is always included regardless of vector distance: if the user already has a `lives_in` fact, a new one must be considered even when Delhi and Berlin embed far apart. The **vector** arm retrieves semantically near facts above a similarity floor of 0.55, catching overlap the exact arm misses — an older `works_at` against a new `works_as`, for instance. The union is passed to the classifier.

The classification. An LLM assigns one of three resolutions:

REINFORCE	the new fact restates a held one; raise its confidence, store no copy
ADDITIVE	the new fact coexists with everything; store it, touch nothing
SUPERSEDE	the new fact makes held facts untrue; store it, expire and link them

The decision that matters is ADDITIVE vs. SUPERSEDE, and it is *not* “same subject and predicate”:

<code>user speaks English + user speaks Hindi</code>	$\rightarrow$ ADDITIVE (multi-valued)
<code>user lives_in Delhi + user lives_in Berlin</code>	$\rightarrow$ SUPERSEDE (single-valued)

Both pairs share a subject and a predicate. Treating that as contradiction would silently delete the user’s second language; treating it as coexistence would leave them living in two cities. Nothing in the schema decides this — it is a judgement about whether a predicate can hold several values at once, which is why a language model makes the call, prompted with explicit single-valued and multi-valued exemplars.

The error policy is deliberately **asymmetric**. A wrong ADDITIVE leaves a stale fact **ACTIVE**, which the retrieval ranking moderates and a later correction repairs. A wrong SUPERSEDE expires something true; the row survives, but it drops out of “what is true now” until someone notices. So the prompt instructs the model to choose ADDITIVE when genuinely unsure. Two further guards run after the model returns: a REINFORCE or SUPERSEDE naming no known memory is downgraded to ADDITIVE (a SUPERSEDE with no valid target would expire nothing while still writing a new fact, producing exactly the contradiction the phase exists to prevent), and a REINFORCE naming several rows is trimmed to the closest one, since reinforcing several at once would inflate confidence across facts the user restated only one of. When the candidate list is empty the model is not called at all — no candidates means no possible conflict, which removes any chance of a hallucinated target on what is also the most common path.

The write. On SUPERSEDE the new memory is inserted and the targets are expired *in the same transaction*: `status = 'EXPIRED'`, `valid_until = now()`, and `superseded_by` set to the new row. Atomicity is required in both directions — a crash between the two statements would either leave two contradictory **ACTIVE** facts or expire the old fact with nothing replacing it. The expiry is guarded on `status = 'ACTIVE'` so a retry cannot overwrite a chain link written by the first attempt. Algorithm 1 states the pipeline.

Algorithm 1 DTSG ingest — from a message to a consistent state graph

```
1: procedure INGEST( $u$ : user,  $m$ : message)
2:    $e \leftarrow \text{INSERTEVENT}(u, m)$   $\triangleright$  own transaction; commits before anything derived
3:    $R \leftarrow \text{EXTRACTFACTS}(m)$   $\triangleright$  LLM, schema-enforced triples
4:    $F \leftarrow \text{NORMALIZE}(R)$   $\triangleright$  subject folding, snake_case, synonym map
5:   for each fact  $f \in F$  do
6:      $v \leftarrow \text{EMBED}(f)$   $\triangleright$  null on failure; degrade, do not drop
7:      $C \leftarrow \text{EXACTMATCH}(u, f) \cup \text{VECTORMATCH}(u, v)$   $\triangleright$  similarity floor 0.55
8:     if  $C = \emptyset$  then
9:        $(r, T) \leftarrow (\text{ADDITIVE}, \emptyset)$   $\triangleright$  no candidates, no possible conflict
10:    else
11:       $(r, T) \leftarrow \text{CLASSIFY}(f, C)$   $\triangleright$  LLM; targets reconciled against  $C$ 
12:    end if
13:    if  $r = \text{REINFORCE}$  then
14:       $\text{BUMPCONFIDENCE}(T_1)$   $\triangleright$  also stamps last_reinforced_at
15:    else
16:      begin transaction
17:       $n \leftarrow \text{INSERTMEMORY}(u, f, v, e)$ 
18:      if  $r = \text{SUPERSEDE}$  then
19:         $\text{EXPIRE}(T, n)$   $\triangleright$  sets superseded_by; guarded on ACTIVE
20:      end if
21:      commit
22:    end if
23:  end for
24: end procedure
```

4.2.2 Temporal retrieval

Retrieval is where the temporal structure is cashed in. Every candidate is scored as

$$\text{score}(f, q, t) = \underbrace{\max(0, \cos(\mathbf{e}_f, \mathbf{e}_q))}_{\text{is it relevant}} \times \underbrace{w(f, t)}_{\text{is it still true}} \times \underbrace{e^{-\lambda \Delta(f, t)}}_{\text{how stale is it}}, \quad (1)$$

where $\mathbf{e}_f$ and $\mathbf{e}_q$ are the fact and query embeddings, t is the reference time of the query, $\Delta(f, t)$ is the age in days, and the status weight is evaluated *against* t rather than read from the stored column:

$$w(f, t) = \begin{cases} w_{\text{exp}}, & \text{if } \text{valid_from}(f) > t \quad (\text{it had not happened yet}) \\ 1.0, & \text{if } \text{valid_until}(f) = \text{NULL} \text{ or } \text{valid_until}(f) > t \\ w_{\text{exp}}, & \text{otherwise (it had already stopped being true)} \end{cases} \quad (2)$$

with $w_{\text{exp}} = 0.15$ by default. The age is measured from a reference instant that differs by status: for an `ACTIVE` fact it is `last_reinforced_at` if the fact has been restated, else `valid_from`; for an `EXPIRED` fact it is `valid_until`, so that among superseded facts the recently changed ones rank highest — which is what makes “what changed lately” answerable by the same formula. The delta is clamped at zero, so a fact timestamped slightly ahead of t (clock skew, or an `as_of` between `valid_from` and now) cannot produce a negative exponent and outrank everything. Cosine similarity is clamped at zero for the same class of reason: it runs $[-1, 1]$, and a negative value would flip the sign of the whole product and turn an irrelevant fact into a top hit.

The decay constant defaults to $\lambda = \ln(2)/180$, a 180-day half-life. Facts people state about themselves age slowly — a city or a job stays true for months or years — so more aggressive

decay buries correct answers faster than they actually go stale. Both λ and w_{exp} are per-request parameters, so a test set can be swept without a redeployment; Section 6 uses exactly this to ablate the terms.

Three modes, one store. Equation (2) is what turns different temporal questions into one query with a different t :

Mode	Eligible rows	Scored at
<code>now</code> (default)	everything; superseded rows discounted	now
<code>as_of</code>	only facts that held at that instant	the given <code>as_of</code>
<code>changes</code>	only rows in a supersede chain, newest transition first	now

Because `as_of` evaluates the status term against that moment, a fact superseded last week counts as fully current for a question about last month. “What did I say before X” therefore becomes a ranking question rather than a separate store.

Two-stage re-ranking. No index can be built over Equation (1), because its terms depend on query time and on caller-supplied constants. So Postgres does the part it is good at — approximate nearest-neighbour search over the HNSW cosine index — and Python applies the full score to the returned pool. The pool is deliberately over-fetched at $6\times$ the requested limit with a floor of 40, because top- K by cosine is *not* top- K by score: a fact that wins on the full formula can sit outside the cosine top- K and would be silently dropped by a tight pool. Each response reports `candidates_considered` so a caller can tell whether the pool saturated, and returns the three terms of Equation (1) separately rather than only their product — a ranking that cannot be decomposed is a ranking that cannot be debugged.

4.2.3 The timeline view

`GET /api/timeline` walks the supersede links into chains, oldest first, and the frontend renders them with current values marked and superseded ones struck through with their validity range (Figure 2). The walk follows `superseded_by` rather than `supersedes`, because that is the complete edge: when one fact invalidates several, each old row records the replacement, while the new row’s single `supersedes` column can only name one of them. A fact that was never superseded comes back as a chain of one, so the UI needs a single rendering path rather than separate “history” and “just a fact” cases. Dangling links (a replacement outside the current page) stop the walk instead of inventing entries, and cycles — which the write path should make impossible — surface their rows rather than hanging or silently dropping them.

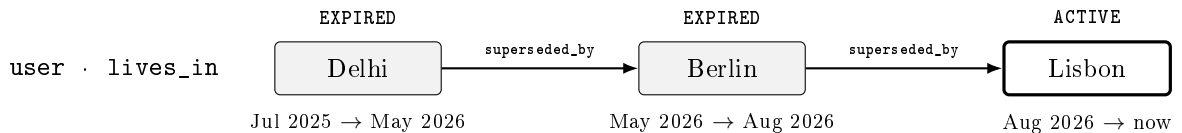


Figure 2: A supersede chain as rendered by the timeline view. No row is ever deleted: two moves produce three rows, of which one is **ACTIVE** and two are **EXPIRED** with closed validity intervals. The chain is walked along `superseded_by`, which is the authoritative edge because a single fact may invalidate several predecessors.

5 Experiment Design

Three experiments were run, each isolating a different variable.

E1 — Policy comparison (DTSG vs. naive RAG). The question is whether temporal ranking beats similarity ranking on the cases where the two can disagree. The two policies are:

- **Baseline (naive RAG):** rank by cosine similarity alone, ties broken by input order, exactly as a database returning rows in index order would. No status weighting, no decay, no re-ranking.
- **DTSG:** rank by Equation (1) with default parameters $w_{\text{exp}} = 0.15$, $\lambda = \ln(2)/180$.

Input is the 4-case set of Table 6; output is the top-ranked fact per case under each policy. Two metrics are recorded: *correct top hit*, counted over the 3 scored cases, and *stale top hit* (the top-ranked fact has status **EXPIRED**), counted over all 4. The second metric matters independently, because a system can be accidentally right about which object to return while still promoting a fact it should know is dead.

The critical design choice is that the cases carry their own similarity scores, so no embedding model runs. Using real embeddings would make the result depend on the embedding model’s quality, which is not the thing under test; by fixing similarity, any difference in outcome is attributable to the ranking policy alone.

On the fairness of the comparison. Both policies read the same **memories** rows, which might look like DTSG is being handed a curated corpus. It is not, and the equivalence is worth stating precisely: a naive store never expires anything, so its corpus is “every fact ever extracted” — and DTSG never deletes anything either, it only marks rows **EXPIRED**. The row sets are identical. The baseline simply ignores the status and time columns, exactly as a system that never wrote them would, which isolates the retrieval policy as the only variable. One caveat runs in the other direction: DTSG’s REINFORCE collapses a restatement into a confidence bump rather than a second row, so a truly naive store would hold a few more near-duplicates than this baseline sees. Duplicates make naive retrieval look more consistent, so the measurement is conservative rather than flattering. The same comparison is available on live data through `POST /api/baseline/compare`, which runs both policies on one query and reports where they diverge.

E2 — Term ablation on a controlled scenario. The question is which term of Equation (1) does the work. A scenario is constructed in which the user moved from Delhi to Berlin 10 days ago and *both facts have cosine similarity exactly 1.000*, so the similarity term is held constant by construction and a vector store cannot distinguish them even in principle. Four configurations are run over the same two rows: (i) **mode=now** with defaults; (ii) $w_{\text{exp}} = 1.0$, which removes the status penalty; (iii) **mode=as_of** 60 days ago, which moves the reference time to before the move; (iv) $\lambda = 0$, which removes decay and isolates the status term. Output for each is the score of both facts with the three terms reported separately.

E3 — Correctness of the pipeline. The question is whether the write path maintains the invariants the design depends on. A suite of 103 automated tests covers extraction and normalization, the classifier’s coercion guards, event immutability and keyset pagination, the insert-and-expire transaction, chain walking including dangling links and cycles, retrieval scoring and re-ranking, and the baseline endpoints. The language-model provider is stubbed throughout, so the suite tests the system rather than the model, and runs without a database or an API key.

Setup. Backend: Python 3, FastAPI, asyncpg, deployed on Render; database: Supabase Postgres with `pgvector`, `vector(1536)` and an HNSW cosine index; models: `gemini-2.5-flash` for extraction and classification, `gemini-embedding-001` at 1536 dimensions for embeddings; frontend: React + TypeScript + Vite on Vercel. E1, E2 and E3 all run offline (`scripts/run_benchmark.py`, `scripts/check_retrieval.py` and `pytest` respectively).

6 Results & Observations

E1 — DTSG vs. naive RAG. Table 7 gives the per-case outcome and Table 8 the aggregate. The separation is complete on this set: the baseline returns a stale fact as its top hit in 3 of 4 cases and gets 0 of 3 scored cases right, while DTSG gets 3 of 3 right and never tops a stale fact.

Table 7: E1 — top-ranked fact per case under each policy. The baseline is wrong in every scored case; note that it is wrong for a different reason each time — once because the stale fact is more similar, once because the two are exactly tied, once because the stale fact is much more similar.

Query	Baseline top hit	DTSG top hit	DTSG correct?
where do I live?	Delhi (EXPIRED)	Berlin (ACTIVE)	yes
where do I work?	Acme (EXPIRED)	Globex (ACTIVE)	yes
what is my job title?	Engineer (EXPIRED)	Staff Engineer (ACTIVE)	yes
which languages do I speak? (control)	English (ACTIVE)	Hindi (ACTIVE)	n/a (either valid)

Table 8: E1 — aggregate. “Correct top hit” is counted over the 3 scored cases; “stale top hit” over all 4.

Metric	DTSG	Naive RAG baseline
Correct top hit $\uparrow$	3/3	0/3
Stale top hit $\downarrow$	0/4	3/4

The control case is the informative one for the opposite failure mode. “Which languages do I speak?” has two **ACTIVE** facts, English and Hindi, with identical similarity; DTSG returns Hindi on top purely because it is more recent, and English remains in the result set. Had the classifier been over-eager and treated **speaks** as single-valued, one of the two would have been expired at write time and this case would have looked like a success in the aggregate while having silently destroyed a true fact. The case is in the set specifically to make that failure visible, and it does not occur.

E2 — which term does the work. Table 9 reports the controlled Delhi/Berlin scenario. Because similarity is pinned at 1.000 for both rows, every difference in the table is produced by the status and recency terms.

Table 9: E2 — term ablation on the Delhi $\rightarrow$ Berlin scenario (moved 10 days ago). Both facts have cosine similarity 1.000 by construction, so a cosine-only ranker sees one number for both rows and cannot choose. Bold marks the top hit.

Configuration	Fact	sim.	status w	recency	score
mode=now (defaults)	Berlin	1.000	1.00	0.962	0.9622
	Delhi	1.000	0.15	0.962	0.1443
$w_{\text{exp}} = 1.0$ (“where have I lived?”)	Berlin	1.000	1.00	0.962	0.9622
	Delhi	1.000	1.00	0.962	0.9622
mode=as_of, -60 days	Delhi	1.000	1.00	1.000	1.0000
	Berlin	1.000	0.15	1.000	0.1500
$\lambda = 0$ (no decay)	Berlin	1.000	1.00	1.000	1.0000
	Delhi	1.000	0.15	1.000	0.1500

Three observations follow. First, **the status term is doing the work**: with decay switched off entirely ($\lambda = 0$), the separation is still 1.0000 vs. 0.1500 — a factor of $1/w_{\text{exp}}$ — whereas the recency term contributes only the common factor 0.962 over a 10-day gap. Decay matters for ordering several live facts (as in the control case of E1), not for separating live from dead. Second, **the discount is a discount, not a filter**: raising w_{exp} to 1.0 brings Delhi back to parity in the same query against the same store, which is what makes “where have I lived?” answerable without a second index. Third, **the ordering inverts under as_of**: asked about a date 60 days ago, Delhi scores 1.0000 and Berlin 0.1500, because Equation (2) is evaluated at the query’s reference time and at that instant Berlin had not happened yet. That inversion is not special-cased anywhere; it falls out of the formula.

E3 — pipeline correctness. All 103 tests pass (103 passed in 1.21s), covering extraction and normalization, the classifier’s target-reconciliation guards, event immutability and keyset pagination, the atomic insert-and-expire, chain walking with dangling links and cycles, the scoring and re-ranking functions, and the baseline endpoints.

What works, and why. The gain in E1 is not an artefact of tuning; it is structural. Cosine similarity measures whether two strings are *about* the same thing, and “I live in Delhi” and “I live in Berlin” are maximally about the same thing — which is exactly why similarity cannot separate them. Any amount of embedding-model improvement makes this worse rather than better, since a better model brings the two closer together. The information needed to choose is not in the text at all; it is in the write-time record of which one replaced the other. DTSG wins these cases because it *stored that record*, and the retrieval formula is only the place where the stored record is spent.

What does not work, and why. Four limitations are visible in these results and should temper them.

1. **The benchmark is small and adversarial by construction.** Four cases, hand-written, all chosen so that similarity alone fails. A 3/3 vs. 0/3 result on such a set demonstrates that the mechanism operates as designed; it does not estimate an accuracy on a realistic query distribution, where most questions have no competing stale fact and both policies would agree. The honest reading is that DTSG is never worse here and is decisively better on the subset that matters, not that it is “100% accurate”.
2. **The upstream stages are not evaluated against a live model.** E1 and E2 hold similarity fixed and E3 stubs the provider, so extraction quality and the classifier’s single- vs. multi-valued judgement are unproven end to end. Everything downstream ranks whatever those two stages write: a missed SUPERSEDE leaves two **ACTIVE** facts and the ranking will faithfully return the wrong one. `scripts/check_extraction.py` and `scripts/check_classifier.py` exist to run these against a real key, and doing so is the first item of future work.
3. **The parameters are chosen, not learned.** $w_{\text{exp}} = 0.15$ and a 180-day half-life are defensible defaults — the first is low enough that a stale fact never outranks a live one on similar text and high enough to stay reachable, the second reflects that self-reported facts age slowly — but neither is fitted to data. They are per-request parameters precisely so that they can be swept once a labelled set exists.
4. **Normalization is a finite table.** 15 canonical predicates and 22 synonyms cover the common cases, but a predicate the table does not know will not match its own earlier variant, and the resulting failure is silent: no conflict is detected, so no supersede occurs,

and the graph accumulates contradictory **ACTIVE** facts that retrieval will rank in good faith.

7 Future Work and Conclusion

Who benefits. Any system that must remember a person across sessions and get the *current* answer right: conversational assistants, customer-support agents holding an account’s history, tutoring systems tracking what a learner has mastered, and health or finance applications where an out-of-date fact is not merely unhelpful but harmful. The immutable log with a scoped erasure hatch is also directly useful for auditability and for handling erasure requests without abandoning append-only semantics elsewhere.

Limitations. Beyond the four listed in Section 6: the classifier makes one language-model call per extracted fact against its candidate set, so ingest cost grows with the number of facts per message rather than with messages; the state graph is flat subject–predicate–object, with no entity resolution, so “Acme” and “Acme Corp.” remain distinct objects; the **supersedes** column holds a single uuid, so when one fact invalidates several the complete edge set lives only on the **superseded_by** side; and there is no consolidation or summarization step, so a very long-lived user accumulates rows indefinitely.

Future work. In rough order of value: (1) **run the live-model checks** and measure extraction precision/recall and classifier agreement against a hand-labelled set — this is the largest open gap; (2) **evaluate on a public benchmark**, specifically the knowledge-update subsets of LongMemEval [7] and LoCoMo [8], which would place these results on the same axis as Zep and Mem0; (3) **sweep w_{exp} and λ** on that labelled set, and consider learning a per-predicate half-life, since a job title and a mood do not age at the same rate; (4) **learn the single-/multi-valued property per predicate** from observed supersede history instead of asking the model each time, which would cut both cost and variance on the decision that matters most; (5) **entity resolution** over objects, so surface variants of one employer or city collapse; (6) **answer generation**, wiring the SMART tier (currently reserved and called by no endpoint) to produce answers grounded in the retrieved, status-annotated facts, with citations to **source_event**; (7) **multi-hop retrieval** over the graph, as in HippoRAG [9], which the current single-hop scoring does not attempt.

Conclusion. We set out to fix a specific and quiet failure: agent memory that answers with a fact that used to be true. The diagnosis is that similarity search has no way to represent the relation between an old fact and the one that replaced it, so the information needed to choose correctly is absent at read time — which is a write-time problem wearing a read-time disguise. DTSG’s response is to record that relation when the contradiction occurs: an immutable event log preserves what was said, a mutable state graph records what is currently believed, and a supersede link joins the two states of every changed fact instead of deleting either. Retrieval then spends that record through one decomposable product of relevance, validity and staleness, whose status term is evaluated at the query’s reference time — so “where do I live?” and “where did I live in the spring?” are the same query with a different parameter, and both are answered from one store. On a benchmark built from exactly the cases where similarity cannot help, this moves the correct top hit from 0/3 to 3/3 and eliminates stale top hits entirely, with a term ablation showing that the status weight rather than recency decay is what produces the separation. The measured gains come from a small adversarial set and the upstream extraction and classification stages remain unvalidated against a live model, so the result should be read as evidence that the mechanism works as designed rather than as an accuracy claim; closing that gap is the immediate next step.

References

- [1] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.
- [2] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- [3] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [4] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.
- [5] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. MemoryBank: Enhancing large language models with long-term memory. *arXiv preprint arXiv:2305.10250*, 2023.
- [6] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*, 2025.
- [7] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024.
- [8] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [9] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [10] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [11] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR)*, 2024.
- [12] Tu Vu, Mohit Iyyer, Xuezhi Wang, Noah Constant, Jerry Wei, Jason Wei, et al. FreshLLMs: Refreshing large language models with search engine augmentation. *arXiv preprint arXiv:2310.03214*, 2023.
- [13] Bhuwan Dhingra, Jeremy R. Cole, Julian Martin Eisenschlos, Daniel Gillick, Jacob Eisenstein, and William W. Cohen. Time-aware language models as temporal knowledge bases. *Transactions of the Association for Computational Linguistics*, 10:257–273, 2022.
- [14] Wenhui Chen, Xinyi Wang, and William Yang Wang. A dataset for answering time-sensitive questions. In *NeurIPS Datasets and Benchmarks Track*, 2021.
- [15] Adam Liška, Tomáš Kočiský, Elena Gribovskaya, Tayfun Terzi, Eren Sezener, Devang Agrawal, et al. StreamingQA: A benchmark for adaptation to new knowledge over time in question answering models. In *International Conference on Machine Learning (ICML)*, 2022.
- [16] Rakshit Trivedi, Hanjun Dai, Yichen Wang, and Le Song. Know-Evolve: Deep temporal reasoning for dynamic knowledge graphs. In *International Conference on Machine Learning (ICML)*, 2017.
- [17] Alberto García-Durán, Sebastijan Dumančić, and Mathias Niepert. Learning sequence encoders for temporal knowledge graph completion. In *Proceedings of EMNLP*, 2018.

- [18] Woojeong Jin, Meng Qu, Xisen Jin, and Xiang Ren. Recurrent event network: Autoregressive structure inference over temporal knowledge graphs. In *Proceedings of EMNLP*, 2020.
- [19] Borui Cai, Yong Xiang, Longxiang Gao, He Zhang, Yunfeng Li, and Jianxin Li. Temporal knowledge graph completion: A survey. *arXiv preprint arXiv:2201.08236*, 2023.
- [20] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [21] Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. Mass-editing memory in a transformer. In *International Conference on Learning Representations (ICLR)*, 2023.
- [22] Richard T. Snodgrass. *Developing Time-Oriented Database Applications in SQL*. Morgan Kaufmann, 1999.